

PL-PS Interfaces

- **AXI4-Lite**

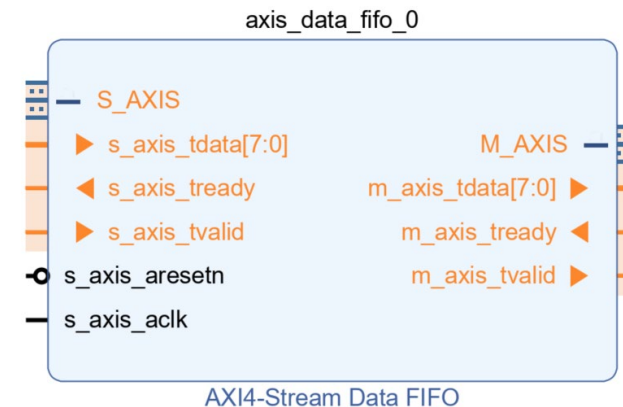
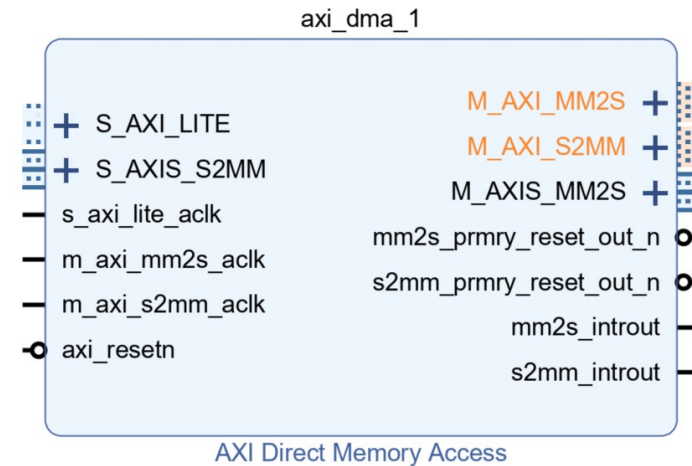
- 标量参数、配置
- 速度最慢

- **AXI4-Master**

- 大量数据（数组、矩阵）
- 可乱序访问

- **AXI4-Stream**

- 大量数据
- 只能顺序访问
- 速度最快



AXI4-Lite

- HLS

- Interface pragma : s_axilite

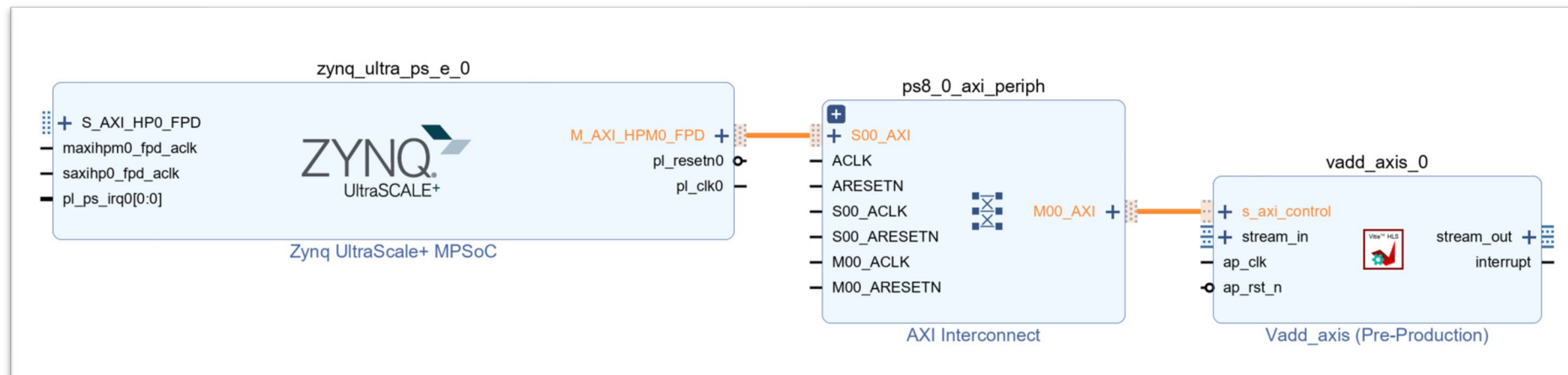
- Block Design

- 连接到PS的M_AXI_HPM
 - 可通过AXI Interconnect 连接多个设备

```
#include "hls_stream.h"
#include "ap_axi_sdata.h"

int sobel(
    hls::stream<axis<ap_uint<8>, 0, 0, 0>>& stream_in,
    hls::stream<axis<ap_uint<8>, 0, 0, 0>>& stream_out,
    int cols,
    int rows
) {

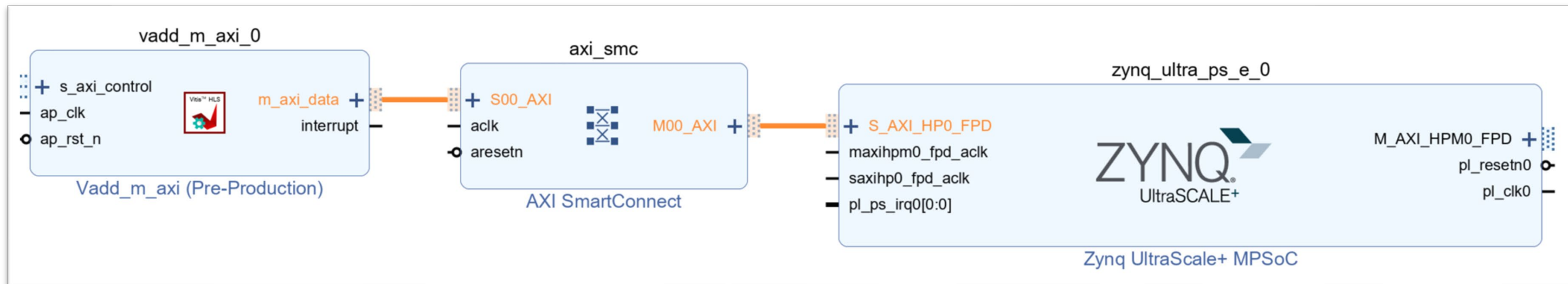
    #pragma HLS INTERFACE axis port=stream_in register_mode=both depth=1024 register
    #pragma HLS INTERFACE axis port=stream_out register_mode=both depth=1024 register
    #pragma HLS INTERFACE s_axilite port=cols
    #pragma HLS INTERFACE s_axilite port=rows
    #pragma HLS INTERFACE s_axilite port=return
}
```



AXI4-Master

- HLS
 - Interface pragma : m_axi
- Block Design
 - 连接到PS的S_AXI_HP
 - 可通过AXI SmartConnect 连接多个设备

```
int sobel(  
    unsigned char* src,  
    unsigned char* dst,  
    int cols,  
    int rows  
) {  
  
#pragma HLS INTERFACE m_axi port=src depth=1024  
#pragma HLS INTERFACE m_axi port=dst depth=1024  
#pragma HLS INTERFACE s_axilite port=cols  
#pragma HLS INTERFACE s_axilite port=rows  
#pragma HLS INTERFACE s_axilite port=return
```



AXI4-Stream

- HLS

- 函数变量: hls::stream
- Interface pragma: axis

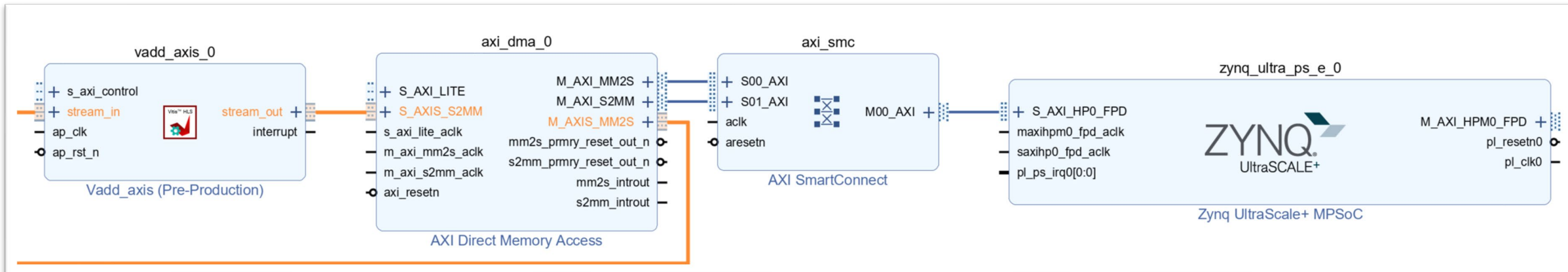
- Block Design

- 连接到AXI DMA的Stream接口
- AXI DMA通过AXI4-Master连接到PS

```
#include "hls_stream.h"  
#include "ap_axi_sdata.h"
```

```
int sobel(  
    hls::stream<axis<ap_uint<8>, 0, 0, 0>>& stream_in,  
    hls::stream<axis<ap_uint<8>, 0, 0, 0>>& stream_out,  
    int cols,  
    int rows  
) {
```

```
#pragma HLS INTERFACE axis port=stream_in register_mode=both depth=1024 register  
#pragma HLS INTERFACE axis port=stream_out register_mode=both depth=1024 register  
#pragma HLS INTERFACE s_axilite port=cols  
#pragma HLS INTERFACE s_axilite port=rows  
#pragma HLS INTERFACE s_axilite port=return
```



solution1\impl\ip\drivers\vadd_m_axi_v1_0\src*_hw.h
或者device_0.register_map

PYNQ APIs

• AXI4-Lite

- device_0.write(data_addr, value)

• AXI4-Master

- buf = allocate(shape=, dtype=)
- device_0.write(data_addr, buf.physical_address)
- device_0.write(0x0, 0x1)

• AXI4-Stream

- buf1 = allocate(shape=, dtype=)
- buf2 = allocate(shape=, dtype=)
- device_0.write(0x0, 0x1)
- dma.sendchannel.transfer(buf1)
- dma.recvchannel.transfer(buf2)
- dma.sendchannel.wait()
- dma.recvchannel.wait()

```
5 // control
6 // 0x00 : Control signals
7 //      bit 0 - ap_start (Read/Write/COH)
8 //      bit 1 - ap_done (Read/COR)
9 //      bit 2 - ap_idle (Read)
10 //      bit 3 - ap_ready (Read)
11 //      bit 7 - auto_restart (Read/Write)
12 //      others - reserved
13 // 0x04 : Global Interrupt Enable Register
14 //      bit 0 - Global Interrupt Enable (Read/Write)
15 //      others - reserved
16 // 0x08 : IP Interrupt Enable Register (Read/Write)
17 //      bit 0 - enable ap_done interrupt (Read/Write)
18 //      bit 1 - enable ap_ready interrupt (Read/Write)
19 //      others - reserved
20 // 0x0c : IP Interrupt Status Register (Read/TOW)
21 //      bit 0 - ap_done (COR/TOW)
22 //      bit 1 - ap_ready (COR/TOW)
23 //      others - reserved
24 // 0x10 : Data signal of a
25 //      bit 31~0 - a[31:0] (Read/Write)
26 // 0x14 : Data signal of a
27 //      bit 31~0 - a[63:32] (Read/Write)
28 // 0x18 : reserved
29 // 0x1c : Data signal of b
30 //      bit 31~0 - b[31:0] (Read/Write)
31 // 0x20 : Data signal of b
32 //      bit 31~0 - b[63:32] (Read/Write)
33 // 0x24 : reserved
34 // 0x28 : Data signal of o
35 //      bit 31~0 - o[31:0] (Read/Write)
36 // 0x2c : Data signal of o
37 //      bit 31~0 - o[63:32] (Read/Write)
38 // 0x30 : reserved
39 // 0x34 : Data signal of size
40 //      bit 31~0 - size[31:0] (Read/Write)
41 // 0x38 : reserved
42 // (SC = Self Clear, COR = Clear on Read, TOW = Toggle on
--
```